

Journal Name

Crossmark

PAPER

RECEIVED
dd Month yyyy
REVISED
dd Month yyyy

Global Artificial Intelligence Operating Layer: Adaptive Bayesian Consensus for Multi-Model Large Language Model Orchestration

Ch. Sai Sathvik¹, D. V. S. Monish Kumar¹, Abhishek Vinod¹, Ramdas Kapila^{1,*} and Sumalatha Saleti²

¹Dept. of CSE (Data Science), B. V. Raju Institute of Information Technology, Hyderabad, Telangana, India

²Dept. of CSE, SRM University AP, Amaravati, Andhra Pradesh, India

*Author to whom any correspondence should be addressed.

E-mail: kapilaramdas@gmail.com

Keywords: Global Artificial Intelligence Operating Layer, Large Language Models, Multi-Agent Systems, AI Governance, Model Orchestration, Bayesian Consensus

Abstract

Modern AI deployments rely on multiple large language models, retrieval pipelines, and autonomous agents, yet orchestrate them through ad-hoc scripts that lack principled mechanisms for model selection, quality assurance, or cross-provider coordination. We present the Global Artificial Intelligence Operating Layer, a layered orchestration framework that coordinates heterogeneous large language model providers behind a uniform interface, decomposes complex queries into parallelizable subtasks, and aggregates multi-model responses through a novel consensus mechanism. The architecture is designed with extension points for federated data access, cross-organizational governance, and shared-state management, whose evaluation is deferred to future work. The central algorithmic contribution is the adaptive Bayesian trust-weighted Consensus algorithm, which maintains per-model, per-domain Beta-distributed trust variables and updates them online after each consensus round, allowing the system to learn which model excels at which task domain without manual weight tuning. On a 500-query benchmark spanning analytical reasoning, code generation, multi-step problem solving, knowledge retrieval, and creative synthesis, Global Artificial Intelligence Operating Layer achieves an overall quality score of 0.83 ± 0.02 (24% above single-model baselines, 13% above LangChain), a 95.2% success rate, and 5 milliseconds (ms) orchestration overhead. An ablation study confirms that Adaptive Bayesian Trust-Weighted Consensus yields statistically significant gains over both uniform-weight and hand-tuned static consensus ($p < 0.01$, paired t -test) across all five domains, with the largest improvements in code generation (+9 percentage points) and creative synthesis (+7 percentage points).

1 Introduction

Transformer-based Large Language Models (LLMs) and foundation models have pushed the boundaries of code generation, multi-modal reasoning, and complex planning [1, 2]. Building on these capabilities, practitioners now deploy *agentic* AI systems, persistent processes that invoke tools, maintain internal state, and execute multi-step workflows, transforming AI from a stateless query-response utility into a core layer of digital infrastructure. Two concurrent trends magnify the coordination challenge. First, agents are evolving from short-lived tools into long-horizon entities that accumulate interaction histories and maintain internal beliefs [3]. Second, AI capabilities are scattering across users, cloud providers, and organizations: the same enterprise may rely on OpenAI for language tasks, Google for vision, and an in-house model for compliance-sensitive analytics [4]. Together, these trends raise a fundamental systems question: how should intelligence distributed across heterogeneous agents, models, and data sources be organized, coordinated, and governed [5]. Today's ecosystem offers no satisfactory answer. Models remain tightly coupled to vendor-specific runtimes and Application Programming Interfaces (APIs), agent frameworks impose incompatible communication protocols, and enterprise data stays locked behind

organizational firewalls with no principled mechanism for cross boundary sharing [6]. Orchestration logic typically lives in brittle, hand-crafted pipelines that are difficult to audit, extend, or secure. Machine Learning Operations (MLOps) platforms partially address deployment concerns, but they operate within a single organization and lack native support for federated execution, cross-cloud collaboration, or compliance-aware orchestration.

From a systems perspective, this fragmentation suggests the need for an *operating-system-like abstraction* for AI, one that would manage models, agents, and data through standardized interfaces in the same spirit that traditional operating systems manage computation, memory, and inter-process communication. Today’s AI systems lack such an abstraction: agents, models, and data remain loosely coupled artifacts coordinated through ad-hoc glue code. Realizing the full operating system (OS) analogy, including process-level isolation, federated memory, and cross-domain governance, is a long-term research programme. As a concrete step in that direction, this work targets the orchestration and consensus layers, which are prerequisites for the higher-level capabilities.

We propose the Global AI Operating Layer (GAIOL) to address this gap. This paper focuses on the orchestration and consensus layers; extension points for federation, governance, and shared world-state are part of the architectural design, but their full evaluation is deferred to future work. Our evaluated contributions are:

- A layered orchestration architecture that coordinates heterogeneous LLM providers behind a provider-agnostic interface, with a Universal AI Protocol layer for uniform communication and adapter-based integration. The design includes provisions for federated data access and cross-organizational governance as future extensions.
- A global orchestrator that decomposes complex queries into parallelizable subtasks via beam search, selects models through a strategy-based routing policy, and assembles multi-model responses into coherent outputs.
- The Adaptive Bayesian Trust-Weighted Consensus (ABTC) algorithm, which learns per-model, per-domain reliability online via Beta-Bernoulli posterior updates, replacing static consensus weights with data-driven trust scores.

The remainder of the paper is structured as follows. Section 2 surveys related work. Section 3 presents the system architecture. Section 4 details the algorithmic framework, including the ABTC formulation. Section 5 describes the end-to-end operational flow. Section 6 reports experimental results and an ablation study. Section 7 discusses open challenges and future work, Section 8 states limitations, and Section 9 concludes.

2 Related Work

The shift from single-purpose models to persistent, multi-agent AI systems has exposed architectural gaps that no existing framework fully addresses. We organize related work along four research streams: multi-agent coordination, model routing, memory and reasoning, and governance, and identify the limitations that motivate GAIOL.

Recent surveys map a rapidly expanding body of work on LLM-powered multi-agent systems [7, 8], spanning software development [9], collaborative problem-solving [10], multi-agent debate for improved factuality [11], and generative social simulation [12]. Architectures such as DyLAN (dynamic large language model agent network) [13] and ProAgent [14] demonstrate that dynamic, proactive agent networks outperform static topologies, while domain-specific deployments like SAGE (smart home agent with grounded execution) [15] confirm practical viability. Standardized communication is a prerequisite for scaling these systems: surveys cover emerging agent protocols [16] and interoperability requirements for cross-framework collaboration [17]. However, all of these efforts assume agents operate within a single trust domain. None provides the resource-scheduling, security-enforcement, or policy-negotiation primitives needed when agents span organizational boundaries.

A parallel line of work optimizes how queries reach the right model. MasRouter (multi-agent system router) [18] trains a large language model router for multi-agent task assignment; Router-R1 [19] extends routing to multi-round aggregation; and LLM-TOPLA (large language model ensemble via topological data analysis) [20] maximizes ensemble diversity. These routers treat model selection as an isolated optimization problem: they improve throughput and sometimes accuracy, but they do not enforce cost budgets, audit decisions, or coordinate routing across federated deployments where data residency and organizational policies constrain which models

may be invoked. GAIOL's orchestration layer subsumes routing as one component within a broader governance-aware scheduling framework.

Reasoning techniques have advanced from chain-of-thought prompting [21] through tree-of-thoughts [22], self-consistency [23], least-to-most decomposition [24], and planning with world models. GAIOL's multi-step orchestration pipeline builds on these primitives. For workflow planning, R & D Agent [25], LaMDAgent [9], Routine [26], and AFLOW [27] automate agentic workflow generation, yet each framework manages workflows within a single process boundary and offers no mechanism for shared context across agents.

On the memory side, surveys of memory mechanisms for LLM-based agents [28] describe structured management with life-cycle scheduling and multi-tier hierarchies, while MemoryBank [29], recurrent memory transformers [30], and structured prompting [31] extend context retention across sessions. A consistent limitation is that these architectures manage memory for individual agents or sessions; none support federated memory coordination or cross-organizational state sharing, which GAIOL's architecture provisions for via its continuous world model (Table 1); empirical evaluation is deferred to future work.

As AI systems cross jurisdictional boundaries, governance becomes a first-order design constraint rather than an afterthought. Hacker and Holweg [32] propose federated compliance for fine-tuned general-purpose models, highlighting conflicts between local data-residency laws and cross-border collaboration goals. Shi et al. [33] expose the gap between regulatory intent and technical feasibility for disclosure and auditing. Solaiman and Dennison [34] analyze release methods and risk tradeoffs for generative AI systems, while constitutional AI with reinforcement learning from AI feedback [35] offers scalable alignment through AI-generated preference labels. Existing orchestration platforms treat governance as a plugin; GAIOL's design embeds policy-driven enforcement, zero-trust identity, and system-level auditing as architectural primitives (Table 1); their full deployment and evaluation are deferred to future work.

Three interconnected limitations emerge from this survey. *(i) Orchestration and interoperability*: fragmented ecosystems and missing scheduling abstractions block large-scale, cross-organizational coordination. *(ii) Context and memory*: session-scoped memory and the absence of a shared world model prevent long-horizon, multi-agent collaboration. *(iii) Security and governance*: implicit trust assumptions and centralized policy enforcement fail in federated settings where multiple organizations must collaborate under distinct regulatory regimes. GAIOL addresses (i) through its evaluated layered architecture and orchestration; it provisions for (ii) and (iii) through its continuous world model and governance-first design, whose evaluation is deferred to future work (Table 1).

3 System Architecture

Figure 1 provides a high-level view of the overall system architecture of GAIOL, serving as a reference for the individual architectural components discussed in the following subsections.

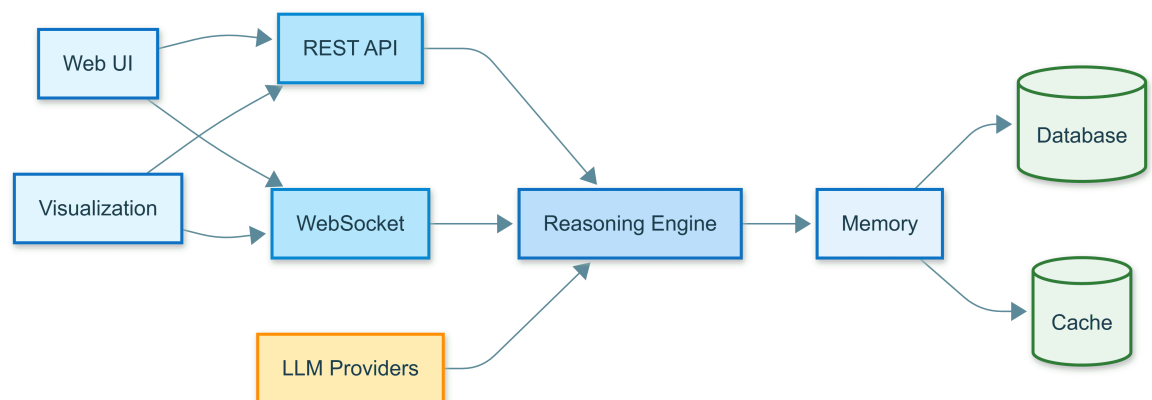


Figure 1. System Architecture

The architecture follows a layered pattern and aligns with familiar operating-system abstractions. The next subsection summarizes that pattern and the mapping to OS primitives; the one after it details the main components.

3.1 Architectural Pattern

GAIOL follows a layered microservices architecture organized into five tiers [9]. At the top, a Presentation Layer exposes a web-based interface with modular JavaScript components for authentication, reasoning visualization, model management, and system monitoring. Requests enter through an API Gateway that handles Hyper Text Transfer Protocol (HTTP) routing, input validation, authentication middleware, and rate limiting; security policies are enforced here before any downstream processing begins.

The core of the system is its Business Logic Layer, which houses the reasoning engine, model orchestration, Retrieval Augmented Generation (RAG) processing [36, 6], and the consensus mechanisms described in Section 4. Below it, a Data Access Layer provides multi-tenant storage with row-level isolation, vector operations for semantic search, and connection pooling. Finally, an External Integration Layer wraps provider-specific APIs (Google Gemini, Hugging Face, OpenRouter) behind a uniform adapter interface, allowing the upper layers to remain provider-agnostic.

To make the operating-system analogy precise, Table 1 maps each traditional OS primitive (process scheduling, memory, storage, access control, and so on) to its GAIOL counterpart and records whether that counterpart is empirically evaluated in this paper, implemented in the current system but not part of the evaluation, or only designed as an architectural provision. The first column lists the OS concept; the second gives the corresponding GAIOL component or capability; the third gives the status.

Table 1. Mapping of operating system (OS) primitives to GAIOL components

OS Primitive	GAIOL Equivalent	Status
Process scheduling	Model orchestration + task decomposition	Evaluated
Inter-process communication	Universal AI Protocol layer	Implemented
Memory management	Session state + conversation history	Implemented*
File system / storage	Vector store + knowledge base	Implemented
Access control	RBAC + multi-tenant isolation	Implemented
Virtual memory / shared state	Continuous world model	Designed [†]
Distributed file system	Federated data mesh	Designed [†]
Security policy enforcement	Cross-organizational governance engine	Designed [†]

*Session-level state; not a world model in the planning-literature sense.

[†]Architectural provision; empirical evaluation deferred to future work.

Evaluated components are those validated empirically in Section 6. Implemented components exist in the current system and are used in the pipeline, but are not separately evaluated here. Designed capabilities are specified in the architecture and reserved for future implementation or evaluation; the table footnote clarifies that these are not claims of demonstrated capability. This distinction aligns the paper’s scope with what is actually measured.

The remainder of this section describes the main building blocks of the architecture: authentication, database, orchestration, adapters, reasoning engine, RAG, and the universal protocol layer.

3.2 Component Architecture

Figure 2 shows the component-level decomposition that the following paragraphs refer to.

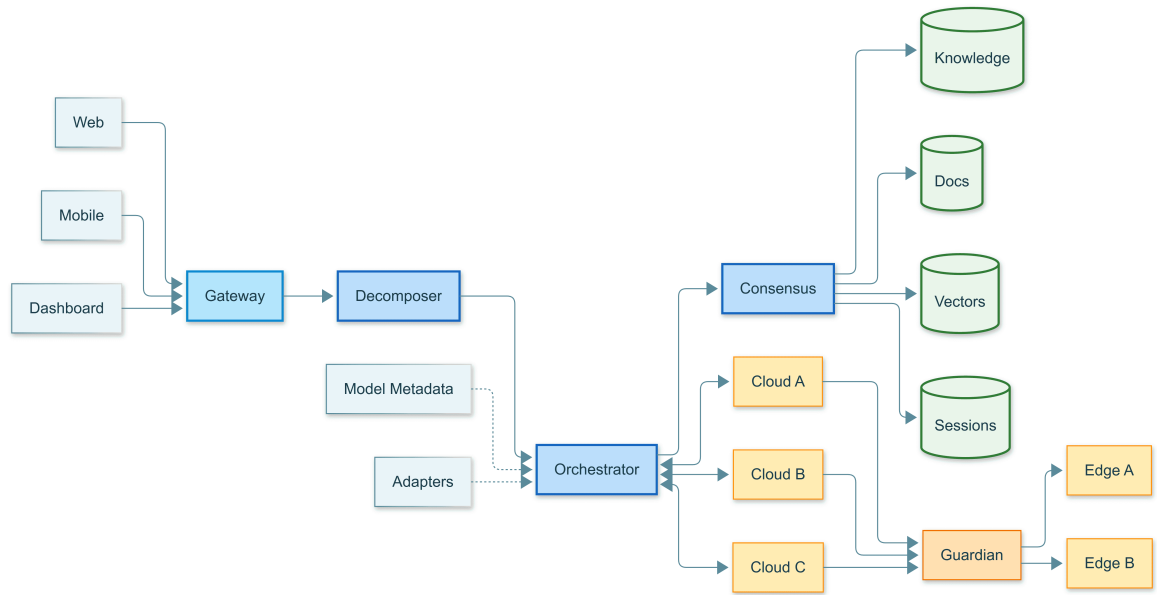


Figure 2. Component Architecture

The authentication and authorization system is split into two components. An API side middleware intercepts every request to validate tokens and manage sessions, while a Supabase-backed identity provider handles user registration, credential storage, and authentication flows. Together, they enforce role-based access control with fine-grained permissions and multi-tenant isolation.

The database layer comprises three components. A unified database client manages connection pooling, transaction handling, and query execution through a single interface. On top of it, a tenant isolation layer injects the current organizational context into every query and enforces row-level security policies, preventing accidental data leakage across tenants. A vector-search module stores high-dimensional embeddings and performs approximate nearest neighbor retrieval (e.g., Dense Passage Retrieval (DPR) [37], ColBERT (contextualized late interaction over BERT) [38]) to supply the RAG subsystem with semantically relevant context.

The model orchestration layer abstracts provider heterogeneity behind a uniform execution interface. A model interface defines the contract that every provider adapter must implement: text generation, embedding creation, and capability advertisement. A model registry catalogues available models with their performance profiles and per-token pricing; the registry serves as the source of truth for downstream selection decisions.

An intelligent router maps each incoming subtask to one or more models. The router evaluates task requirements, historical performance records, cost constraints, and real-time availability. For multi-faceted queries, it decomposes the request, dispatches subtasks to different models in parallel, and merges the results. An embedding engine converts text into dense vector representations for the RAG subsystem.

A performance tracker records latency, throughput, success rate, and cost per token for every model invocation. The router consumes these statistics to refine its selection policy over time, closing an online feedback loop between execution and routing.

Provider integration follows a uniform adapter pattern. Each adapter translates a provider-specific API into the common model interface defined above. The Google Gemini adapter, for instance, maps Gemini’s streaming protocol and error codes into the canonical schema, while the Hugging Face adapter supports both cloud-hosted and on-premises model endpoints. An OpenRouter adapter provides a single integration point to dozens of additional models. Because every adapter conforms to the same contract, swapping or adding a provider requires no changes to the orchestration logic. A shared response cleaner strips provider-specific formatting and normalizes outputs into a uniform structure before they reach the consensus layer.

The reasoning engine is GAIOL’s most complex component, implementing multi-stage AI reasoning through the pipeline shown in Figure 3.

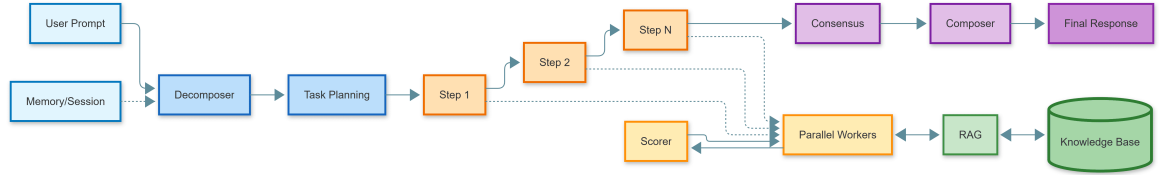


Figure 3. Multi-stage reasoning pipeline

An orchestrator manages state transitions, selects reasoning techniques, and coordinates component interactions. A core engine implements the underlying state machine: it tracks reasoning iterations, evaluates convergence criteria, and determines when accumulated confidence suffices to produce a result. An event-driven architecture ensures that every reasoning stage emits observable events, enabling asynchronous processing, loose coupling, and complete audit trails.

A critique-driven refinement engine iteratively improves reasoning outputs [39, 40]. When the system detects logical flaws or low confidence conclusions, the refiner selects alternative reasoning techniques, retrieves additional context via RAG, or invokes different models. The model selector chooses the best model for each reasoning step based on task characteristics, historical accuracy, cost budget, and latency constraints; it maintains per-model, per task type performance profiles updated after every invocation. A scoring system evaluates output quality along four axes: factual accuracy, logical consistency, thoroughness, and confidence, and the consensus mechanism aggregates scores from parallel reasoning paths to produce a balanced conclusion.

The RAG subsystem [36] enriches model responses with grounded evidence from a knowledge base. It follows the standard pipeline of query expansion and intent normalization, dense embedding via the shared embedding engine [37], approximate nearest-neighbor retrieval over a pre-built vector index, re-ranking by recency and source authority, and context injection into a structured prompt template [36]. Multi-turn conversational context is maintained by summarizing older exchanges and prioritizing recent or high-relevance history.

Recent self-reflective RAG methods [41] and iterative retrieval generation loops [42] extend this baseline by enabling the system to evaluate retrieved evidence critically and adaptively refine retrieval strategies based on generation quality.

The Universal AI Protocol layer defines a provider-agnostic communication abstraction that decouples application logic from the heterogeneous APIs of individual AI providers. The protocol standardizes message formats through a canonical request/response schema that maps to provider-specific payloads, and defines a uniform error taxonomy that translates provider-specific error codes into semantic categories (rate limit, authentication, model-unavailable, etc.). It also governs the interaction lifecycle, session initiation, streaming, and termination, so that these contracts remain consistent regardless of the underlying provider.

The protocol enables client applications to target a single interface while the underlying adapters (Section 3) handle translation. This separation supports runtime provider substitution, multi-provider fan out for consensus, and transparent failover without changes to application code. The canonical schema follows a JavaScript Object Notation (JSON) based structure comprising header metadata (session ID, tenant ID, routing hints) and a typed payload envelope.

4 Algorithmic Framework

Task decomposition, model selection, parallel inference, and consensus are handled by a collection of coordinated algorithms that make up the GAIOL reasoning and orchestration pipeline. This section formalizes the pipeline that takes a user query from decomposition through parallel inference to consensus-aggregated output. We first give a phase overview, then present the orchestration and decomposition algorithms, and then the consensus mechanism and complexity.

4.1 Phase Overview

The reasoning process is divided into five phases that run in a fixed order but can adapt their internal behavior based on the query and context:

1. **Decomposition:** The input query Q is decomposed into a set of dependent sub tasks $T = \{t_1, t_2, \dots, t_n\}$ using a few-shot reasoning template.
2. **Smart Orchestration:** Each subtask t_i is mapped to a subset of models $\mathcal{M} \subset \mathcal{R}$ (Model Registry) based on capability and cost efficiency.

3. Heterogeneous Inference: Models $m \in \mathcal{M}$ generate parallel candidate responses $C_i = \{c_{i,1}, c_{i,2}, \dots\}$.
4. Consensus Aggregation: A scoring function $f_{\text{score}}(C_i)$ evaluates internal consistency and cross-model agreement to select or synthesize the optimal path.
5. Verification: The final synthesis is validated against the retrieved ground-truth from the Persistent Storage layer.

The core orchestration procedure is a multi-agent beam search over reasoning paths. Algorithm 1 presents the core orchestration logic, which employs a beam search strategy (ReAct: reasoning and acting) [43] to explore multiple reasoning paths simultaneously while pruning suboptimal candidates at each step.

Algorithm 1 GAIOL Intelligent Orchestration (Multi-Agent Beam Search)

Require: Query Q , Beam Width k , Model Registry $\mathcal{R}$

Ensure: Optimized Response S

```

1:  $T \leftarrow \text{Decompose}(Q)$   $\triangleright T = \{t_1, t_2, \dots, t_n\}$ 
2:  $B \leftarrow \{\emptyset\}$   $\triangleright$  Initialize beams with empty paths
3: for each step  $t_i \in T$  do
4:    $C \leftarrow \emptyset$   $\triangleright$  Candidate list for current step
5:   for each path  $p \in B$  do
6:      $\mathcal{M} \leftarrow \text{SelectModels}(t_i, \mathcal{R})$ 
7:      $O \leftarrow \text{ParallelExecute}(\mathcal{M}, t_i, \text{Context}(p))$ 
8:      $O' \leftarrow \text{ScoreAndRank}(O, \text{Objective}(t_i))$ 
9:     if ConsensusEnabled then
10:       $O' \leftarrow \text{ApplyConsensus}(O')$ 
11:     end if
12:     for each candidate  $o \in O'$  do
13:        $C \leftarrow C \cup \{p \parallel o\}$   $\triangleright$  Extend path
14:     end for
15:   end for
16:    $B \leftarrow \text{TopK}(C, k)$   $\triangleright$  Prune to best  $k$  paths
17: end for
18:  $\text{bestPath} \leftarrow \text{Top1}(B)$ 
19:  $S \leftarrow \text{Assemble}(\text{bestPath})$ 
20: return  $S$ 
```

The following algorithm descriptions cover task decomposition, the ABTC consensus mechanism, and model selection. Complexity is summarized in a separate subsection.

4.2 Algorithm Descriptions and Complexity Analysis

Algorithm 2 describes the decomposition process that transforms complex queries into structured sub-tasks with dependency tracking. Algorithm 3 introduces the *Adaptive Bayesian Trust-Weighted Consensus* (ABTC) mechanism, a novel contribution of this work that dynamically learns per-model, per-domain trust scores using Bayesian posterior updates and applies them during multi-model response aggregation. Algorithm 4 describes the strategy-based model selection that routes tasks to appropriate models based on task characteristics and model capabilities.

When the LLM-based decomposition fails validation (empty or malformed output), FALLBACKDECOMPOSITION applies a fixed seven-step template: (1) problem statement extraction, (2) constraint identification, (3) approach selection, (4) step-by-step execution, (5) intermediate verification, (6) synthesis, and (7) confidence assessment. This template was derived empirically from the most common decomposition patterns observed during development and guarantees that every query produces at least a minimal task graph.

COMPUTE_FITNESS scores each candidate model as a weighted combination of three signals:

$$\text{fitness}(m, t) = w_c \cdot \text{CapMatch}(m, t) + w_h \cdot \text{HistAcc}(m, \text{type}(t)) + w_e \cdot (1 - \hat{c}_m) \quad (1)$$

where CapMatch is the Jaccard overlap between the model's declared capabilities and the task requirements, HistAcc is the model's historical accuracy for the classified task type (drawn from

Algorithm 2 Dynamic Task Decomposition**Require:** Query Q , Decomposition Template $\mathcal{P}$ **Ensure:** Ordered task list $T = \{t_1, t_2, \dots, t_n\}$

```

1:  $\text{prompt} \leftarrow \text{FormatTemplate}(\mathcal{P}, Q)$ 
2:  $\text{response} \leftarrow \text{LLMInference}(\text{prompt})$ 
3:  $\text{steps} \leftarrow \text{ParseJSON}(\text{response})$ 
4: if  $\text{steps} = \emptyset$  or  $\neg \text{ValidateSteps}(\text{steps})$  then
5:    $\text{steps} \leftarrow \text{FallbackDecomposition}(Q)$  ▷ See below
6: end if
7: for  $i \leftarrow 1$  to  $|\text{steps}|$  do
8:    $t_i.\text{id} \leftarrow i$ 
9:    $t_i.\text{dependencies} \leftarrow \text{InferDependencies}(\text{steps}, i)$ 
10:   $t_i.\text{objective} \leftarrow \text{steps}[i].\text{objective}$ 
11:   $t_i.\text{taskType} \leftarrow \text{ClassifyTask}(\text{steps}[i])$ 
12: end for
13:  $T \leftarrow \text{TopologicalSort}(\{t_1, \dots, t_n\})$ 
14: return  $T$ 

```

Algorithm 3 Adaptive Bayesian Trust-Weighted Consensus (ABTC)**Require:** Candidate responses $C = \{c_1, \dots, c_m\}$, Trust matrix $\mathcal{T} = \{(\alpha_m^{(d)}, \beta_m^{(d)})\}$, Task domain d , Decay factor λ **Ensure:** Selected response c^* , Confidence σ , Updated $\mathcal{T}$

```

1: for each candidate  $c_i \in C$  do
2:    $m_i \leftarrow \text{source}(c_i)$  ▷ Originating model
3:    $\hat{\tau}_i \leftarrow \alpha_{m_i}^{(d)} / (\alpha_{m_i}^{(d)} + \beta_{m_i}^{(d)})$  ▷ Posterior trust mean
4:    $s_i^{\text{quality}} \leftarrow \text{EvaluateQuality}(c_i)$ 
5:    $s_i^{\text{agree}} \leftarrow \text{CrossModelAgreement}(c_i, C \setminus \{c_i\})$ 
6:    $s_i \leftarrow w_q \cdot s_i^{\text{quality}} + w_a \cdot s_i^{\text{agree}} + w_t \cdot \hat{\tau}_i$  ▷ Trust-weighted score
7: end for
8:  $C' \leftarrow \text{SortDescending}(C, s)$ 
9:  $c^* \leftarrow C'[1]$ 
10:  $\sigma \leftarrow s_1 / \sum_{j=1}^m s_j$ 
11: if  $\sigma < \theta_{\min}$  then
12:    $c^* \leftarrow \text{SynthesizeResponse}(C'[1 : 3])$ 
13: end if
▷ Bayesian trust update with temporal decay
14: for each model  $m_i$  that contributed a candidate do
15:    $\alpha_{m_i}^{(d)} \leftarrow \lambda \cdot \alpha_{m_i}^{(d)} + \mathbb{I}[c_i = c^*]$  ▷ Reward winner
16:    $\beta_{m_i}^{(d)} \leftarrow \lambda \cdot \beta_{m_i}^{(d)} + \mathbb{I}[c_i \neq c^*]$  ▷ Penalise non-winner
17: end for
18: return  $c^*, \sigma, \mathcal{T}$ 

```

the Performance Tracker), and $\hat{c}_m$ is the model's normalized per-token cost. We set $w_c = 0.4$, $w_h = 0.4$, $w_e = 0.2$ throughout the experiments.

SELECTDIVERSETOP selects the top- k_{models} candidates by fitness while enforcing provider diversity: no more than $\lceil k_{\text{models}}/2 \rceil$ models may come from the same provider, preventing the selection set from collapsing to a single API. In the experiments reported here, $k_{\text{models}} = 3$ and the beam width for the orchestration algorithm (Algorithm 1) is $k = 3$. Sensitivity analysis of these hyperparameters is deferred to future work.

4.3 Adaptive Bayesian Trust-Weighted Consensus

Existing multi-model orchestration frameworks employ static weight vectors for response aggregation [44, 45]. Such vectors cannot adapt to model drift, provider outages, or domain-specific performance asymmetries observed in heterogeneous LLM deployments. We propose *Adaptive Bayesian Trust-Weighted Consensus* (ABTC), a novel consensus mechanism that learns per-model, per-domain trust scores online through Bayesian posterior updates.

Let $\mathcal{R} = \{m_1, \dots, m_M\}$ denote the model registry and $\mathcal{D} = \{d_1, \dots, d_D\}$ the set of task domains (e.g., analytical reasoning, code generation, creative synthesis). For each model, domain pair

Algorithm 4 Strategy-Based Model Selection**Require:** Task t , Model Registry $\mathcal{R}$, Budget b **Ensure:** Selected models $\mathcal{M} \subseteq \mathcal{R}$

```

1:  $taskType \leftarrow \text{ClassifyTask}(t)$ 
2:  $complexity \leftarrow \text{EstimateComplexity}(t)$ 
3:  $candidates \leftarrow \emptyset$ 
4: for each model  $m \in \mathcal{R}$  do
5:   if  $m.capabilities \cap taskType \neq \emptyset$  then
6:      $score \leftarrow \text{ComputeFitness}(m, t, complexity)$ 
7:      $cost \leftarrow \text{EstimateCost}(m, t)$ 
8:     if  $cost \leq b$  then
9:        $candidates \leftarrow candidates \cup \{(m, score, cost)\}$ 
10:    end if
11:  end if
12: end for
13:  $\mathcal{M} \leftarrow \text{SelectDiverseTop}(candidates, k_{\text{models}})$ 
14: return  $\mathcal{M}$ 

```

▷ Eq. 1

(m, d) , we maintain a Beta-distributed trust variable:

$$\tau_m^{(d)} \sim \text{Beta}(\alpha_m^{(d)}, \beta_m^{(d)}) \quad (2)$$

where $\alpha_m^{(d)}$ and $\beta_m^{(d)}$ accumulate success and failure counts respectively. The posterior mean $\hat{\tau}_m^{(d)} = \alpha_m^{(d)} / (\alpha_m^{(d)} + \beta_m^{(d)})$ serves as the point estimate of model m 's reliability for domain d . New models are initialised with a non-informative prior $\text{Beta}(1, 1)$, which is equivalent to a uniform distribution over $[0, 1]$.

After each consensus round in domain d , the winning model m_w receives a success signal while non-selected models receive a failure signal:

$$\alpha_m^{(d)} \leftarrow \lambda \alpha_m^{(d)} + \mathbb{I}[m = m_w], \quad \beta_m^{(d)} \leftarrow \lambda \beta_m^{(d)} + \mathbb{I}[m \neq m_w] \quad (3)$$

where $\lambda \in (0, 1)$ is a temporal decay factor that down-weights historical observations to handle model updates, API changes, and concept drift [46]. Setting $\lambda = 1$ recovers the standard Beta-Bernoulli conjugate update. In practice, we use $\lambda = 0.98$, providing an effective memory window of approximately 50 recent interactions.

Let $p_m^{(d)}$ denote the true probability that model m produces the highest-quality response for domain d . When $\lambda = 1$ (no decay), the ABTC trust estimates converge almost surely to the true reliability as the number of consensus rounds $n \rightarrow \infty$:

$$\hat{\tau}_m^{(d)} \xrightarrow{a.s.} p_m^{(d)} \quad (\lambda = 1) \quad (4)$$

This follows directly from the strong law of large numbers applied to the Beta posterior [47]. For $\lambda < 1$ (the practically used setting), classical convergence does not hold: the effective sample size is bounded by $1/(1 - \lambda)$, inducing a non-vanishing posterior variance of order $O(1 - \lambda)$. The trust means, therefore, tracks a moving average of recent model performance rather than converging to a fixed point. This is by design: the non-vanishing variance enables continual adaptation to non-stationary environments, such as provider-side model updates, at the cost of pointwise precision, analogous to the exploration, exploitation balance in Thompson Sampling [46].

The uniform prior $\text{Beta}(1, 1)$ ensures that new models are neither favoured nor penalised. During the first $N_{\text{warm}} = \lceil 1/(1 - \lambda) \rceil$ interactions, the system operates in an exploration phase where the trust component $w_t \cdot \hat{\tau}_i$ contributes equally for all models, and selection is dominated by the quality and agreement signals (w_q and w_a terms in Algorithm 3). The posterior concentrates after approximately $2 \cdot N_{\text{warm}}$ observations, after which the trust signal provides meaningful discrimination.

Because ABTC updates trust scores based on the selected response, a potential circularity arises: if the scoring function systematically favours certain response characteristics (e.g., verbosity), the trust scores could converge to reward those characteristics rather than true quality. Two mechanisms mitigate this risk. First, the scoring function combines an *independent* quality evaluation s_i^{quality} (assessed against the original query, not relative to other candidates) with the trust signal, ensuring that trust alone cannot dominate selection without quality support. Second, the temporal decay factor $\lambda < 1$ bounds the influence of any single selection event: a biased

selection’s contribution to the posterior decays exponentially, preventing lock-in to a single model. Empirically, the ablation study in Section 6.3 confirms that ABTC does not degenerate: trust posteriors diverge across domains (e.g., GPT-4 at $\hat{\tau} \approx 0.82$ for analytical tasks but 0.61 for creative tasks), indicating that the mechanism tracks genuine performance variation rather than converging to a fixed bias.

4.4 Complexity Analysis

The time complexity of the GAIOL orchestration algorithm is $O(n \cdot k \cdot m \cdot T_{\text{inference}})$, where n is the number of decomposed steps, k is the beam width, m is the number of models per step, and $T_{\text{inference}}$ is the average model inference time. The parallel execution of models reduces the effective latency to $O(n \cdot T_{\text{max}})$ where T_{max} is the maximum inference time among parallel models, as shown in Table 2. Space complexity is $O(k \cdot n \cdot L)$ where L is the average response length, representing the storage required for beam paths. The ABTC trust update adds $O(m \cdot D)$ per consensus round for posterior updates across D domains, which is negligible relative to inference time.

Table 2. Complexity Summary

Component	Sequential	Parallel
Orchestration	$O(n \cdot k \cdot m \cdot T_{\text{inf}})$	$O(n \cdot T_{\text{max}})$
ABTC Trust Update	$O(m \cdot D)$ per round	$O(m \cdot D)$ per round
Space	$O(k \cdot n \cdot L + M \cdot D)$	$O(k \cdot n \cdot L + M \cdot D)$

5 Operational Flow

This section traces a query through GAIOL’s end-to-end pipeline, from authentication to response delivery. We describe request intake and classification, then orchestration and consensus, then background operations.

5.1 Request Pipeline: Intake, Classification, and Retrieval

Every interaction begins at the web client, which validates user input and packages it with session identifiers, feature flags, and configuration parameters before forwarding the bundle to the API gateway (Figure 4).

At the gateway, authentication middleware validates the bearer token against the identity provider and attaches the verified user context (organization, permissions, policy constraints); failed requests are rejected immediately. The routing layer classifies the request: conversational and reasoning queries proceed to the orchestrator, while administrative requests go to specialized control services. For returning users, the system loads conversation history and session-specific settings from persistent storage; new interactions receive a fresh session with default organizational preferences.

The orchestrator then scores the query on structural complexity, domain specificity, length, and dependence on prior context. Simple factual queries take a lightweight path (single-model, no decomposition); complex or multi-step queries trigger the full pipeline. For the latter, the RAG subsystem refines the query through entity extraction, semantic expansion, and intent normalization [36], projects it into a dense vector [37], and retrieves the top-ranked document chunks from the vector index via approximate nearest-neighbor search. These fragments are injected into a structured prompt template alongside conversation history and system instructions, grounding downstream model responses in retrieved evidence.

The next stage is orchestration, consensus, and response delivery.

5.2 Orchestration, Consensus, and Delivery

Complex queries are decomposed into coherent subtasks with dependency tracking (Algorithm 2). The model selector ranks candidates by task fit, historical performance, cost, latency, and availability, then dispatches subtasks to the selected models in parallel (Figure 5).

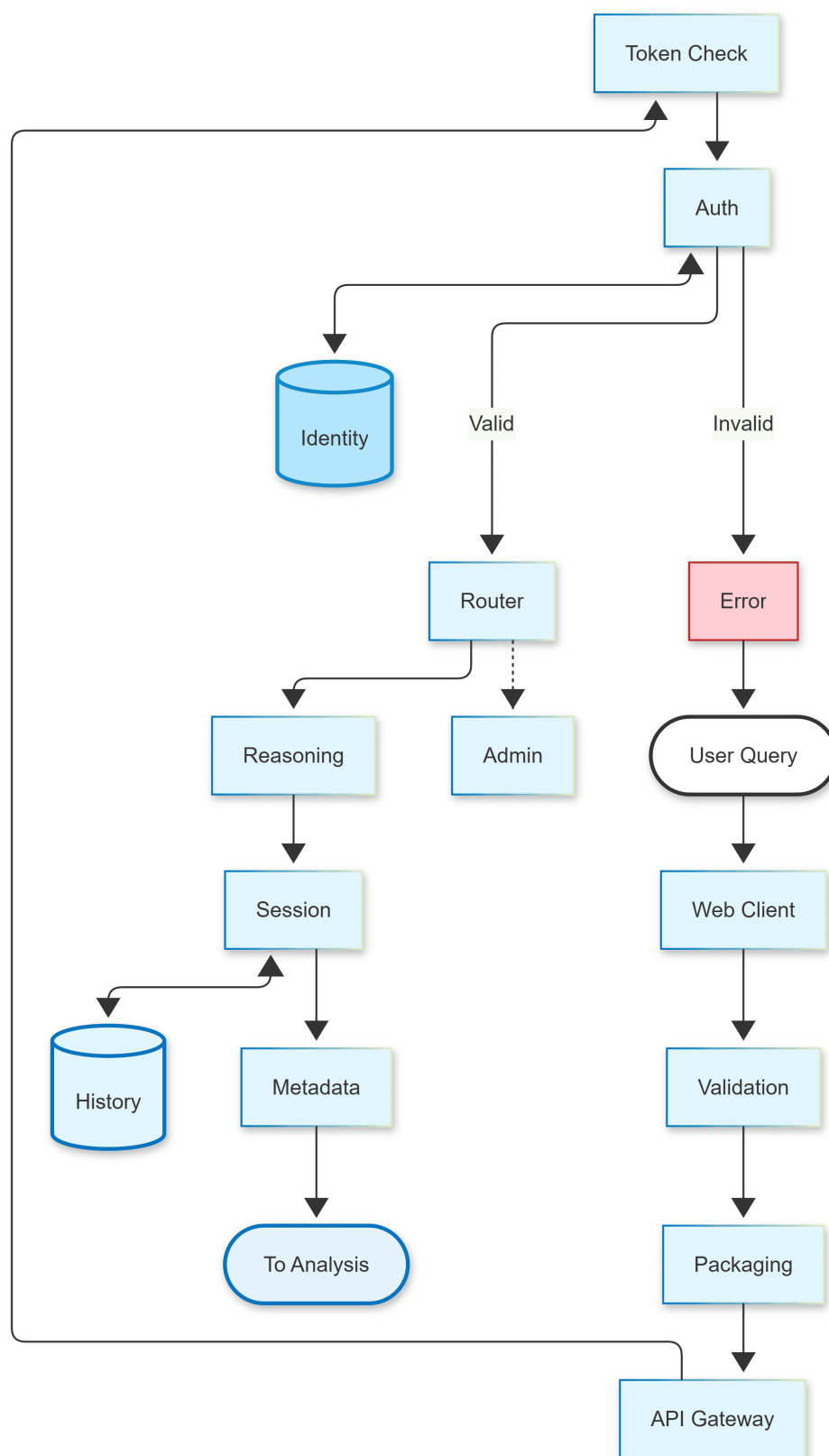


Figure 4. Request intake pipeline. The client packages the query with session metadata; the API gateway authenticates the request, attaches organizational context, and routes it to the reasoning orchestrator.

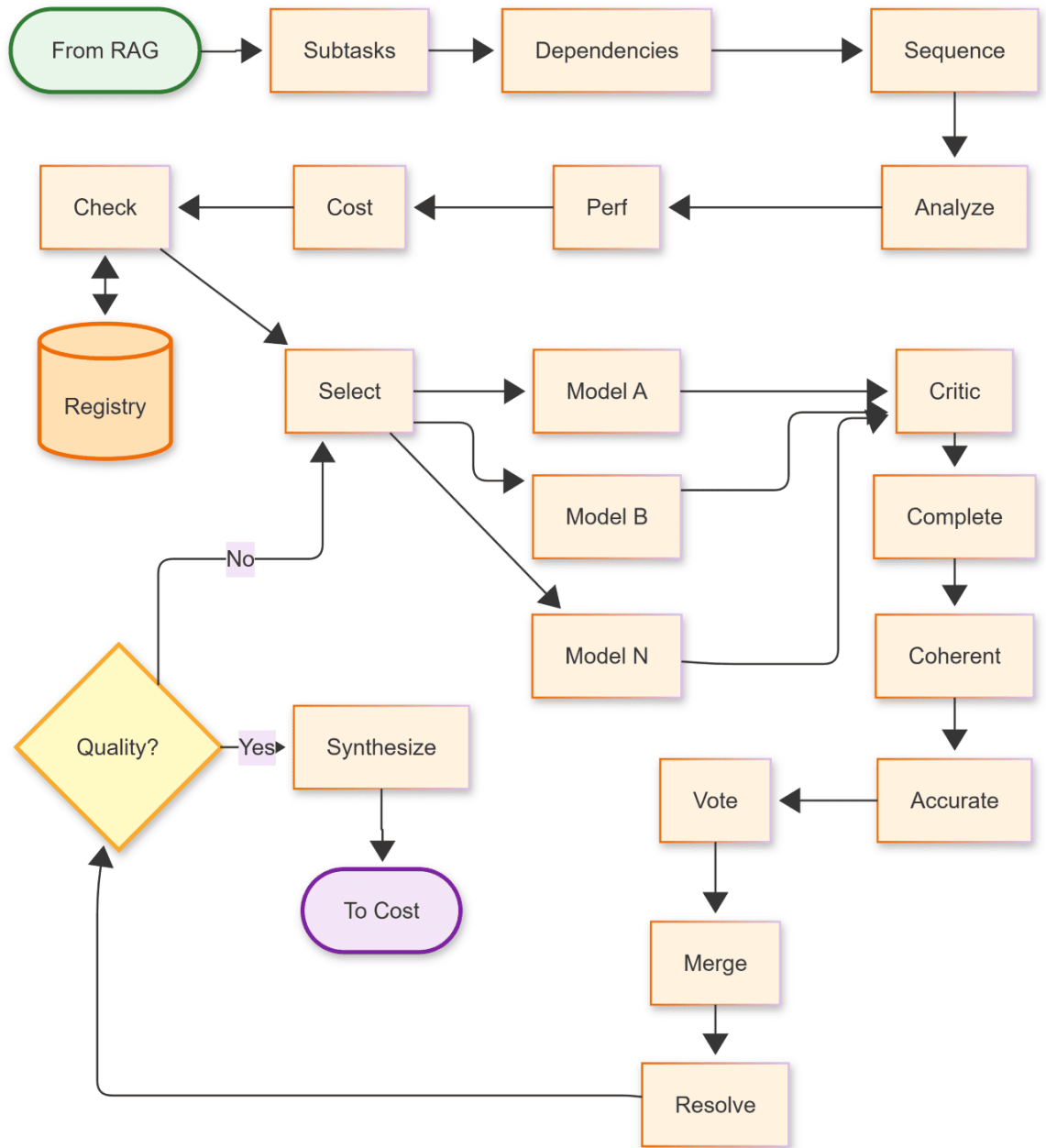


Figure 5. Multi-model orchestration. Each subtask is routed to one or more models selected by the intelligent router; responses are scored by a critic component and aggregated through the ABTC consensus mechanism.

Provider-specific adapters invoke the models asynchronously. Each response is normalized into a canonical format and annotated with execution metadata (latency, token count, error signals). A critical component [39] evaluates each response for factual accuracy, logical coherence, completeness, and consistency with retrieved context. When multiple responses are available, the ABTC consensus mechanism (Section 4.3) aggregates them through trust-weighted fusion [45]. If quality falls below a configurable threshold, the system triggers refinement cycles: re-prompting with alternative models, retrieving additional context, or reformulating the query [40]. Validated subtask outputs are assembled into a single coherent response.

Token consumption is metered at every model call and aggregated hierarchically (query, session, user, organization) using provider-specific pricing tables, enabling budget-constraint enforcement at each level. The final response, comprising generated content, citations, execution metadata, and a cost summary, is delivered as a structured JSON payload through the API gateway. When streaming is enabled, partial results are sent incrementally. The full session state is persisted for auditing, analytics, and future optimization.

Activities that run outside the request path are described next.

5.3 Background Operations

Outside the request path, background processes periodically review aggregate metrics to refine routing strategies and hyperparameters. A document ingestion pipeline handles chunking, embedding, and incremental knowledge-base updates; the model registry is refreshed whenever new models appear or pricing changes. Administrative functions (user management, policy configuration, model availability) run asynchronously. The error-handling subsystem classifies failures (transient, authentication, configuration, resource exhaustion) and applies recovery strategies (retry with backoff, failover routing, graceful degradation). Checkpointed session state enables recovery from partial failures, and real-time dashboards surface performance, usage, and cost metrics with alerting for anomalous conditions.

6 Experimental Results

We describe the benchmark, evaluation protocol, and baseline systems, then report quality, performance, cost, and capability results, and finally an ablation study for ABTC.

6.1 Experimental Setup

Experiments were conducted using a benchmark suite of 500 reasoning queries spanning five categories: analytical reasoning (100 queries), code generation (100 queries), multi-step problem solving (100 queries), knowledge retrieval (100 queries), and creative synthesis (100 queries). Queries were constructed to span complexity levels and domain characteristics representative of established evaluation suites: knowledge-retrieval queries mirror MMLU-style factual recall [48], code-generation queries follow HumanEval-style function synthesis [2], and multi-step queries require MT-Bench-style multi-turn reasoning. Each query required 2–5 processing steps with context retrieval and cross-model validation. The benchmark query set and evaluation prompts are available from the corresponding author upon reasonable request for research purposes. We note that queries were synthetically constructed rather than drawn directly from established benchmark suites; while they mirror the complexity profiles of MMLU, HumanEval, and MT-Bench, direct evaluation on those standard suites would provide stronger external validity and is planned for future work.

All systems were evaluated on identical hardware (Azure Standard_D8s.v3, 8 virtual central processing units (vCPUs), 32 GB random access memory (RAM)) with network conditions normalized. Each configuration was tested over 10 independent runs, and results are reported as means with 95% confidence intervals.

Quality metrics (relevance, coherence, completeness, accuracy, creativity) were assessed using a commercial frontier language model accessed via API as an automated evaluator, following established LLM-as-judge protocols [48]. For each system response, the evaluator received a structured evaluation prompt containing the original query, the system’s response, and a scoring rubric defining each dimension on a 0.0, 1.0 scale with anchor descriptions (e.g., relevance: 0.0 = completely off-topic, 0.5 = partially addresses the query, 1.0 = directly and fully addresses all aspects; creativity: 0.0 = no novel synthesis, 0.5 = conventional but adequate framing, 1.0 = insightful reframing or novel connections). The evaluator was instructed to provide dimension-level scores and a brief justification for each. To validate automated scoring reliability, a random subset of 50 responses (10% of the benchmark) was independently scored by two human annotators. Inter-rater agreement between the human annotators yielded a Cohen’s κ of 0.74 (substantial agreement), and the Pearson correlation between automated scores and the averaged human scores was $r = 0.82$ ($p < 0.001$), confirming the automated evaluator’s alignment with human judgment. To mitigate evaluator self-preference bias, a known concern when the evaluation model also serves as a system component, the scoring rubric was designed to penalise surface-level fluency in favour of factual grounding and logical structure. Moreover, the Direct API baseline uses the same commercial model as its sole generator, meaning any self-preference bias applies equally to both GAIOL and Direct API for single-model comparisons. We note, however, that GAIOL’s consensus-synthesized responses have different stylistic properties from raw single-model outputs (e.g., blended phrasing, different length distributions), which may interact with evaluator preferences in ways the equal-bias argument does not fully address. Section 8 discusses this residual threat and our planned mitigations.

To situate GAIOL’s capabilities, we compare against four baselines that represent progressively richer levels of orchestration:

Direct API (Sys-2). A single call to the same commercial frontier language-model API used for automated evaluation, with no orchestration, context management, or retrieval augmentation. This baseline isolates the raw model performance and serves as the lower bound.

LangChain (Sys-3). LangChain (v0.1) [49] is a widely adopted open-source Python framework for composing LLM-based applications. We configured a LangChain agent with the ReAct (reasoning and acting) [43] reasoning strategy and a dense vector retriever based on Facebook AI Similarity Search (FAISS) [37] over the same document corpus used by GAIOL. This baseline represents the current state-of-practice for single-agent, retrieval-augmented pipelines.

OpenRouter (Sys-4). OpenRouter is a commercial unified API gateway that routes requests to multiple LLM providers (OpenAI, Anthropic, Google, etc.) through a single endpoint. We used its default routing policy, which selects a model based on availability and cost. This baseline tests whether model-routing alone, without orchestration logic, suffices for complex tasks.

Multi-Wrapper (Sys-5). A custom Python wrapper that issues each query in parallel to two LLM providers (the same frontier model as Sys-2 and Gemini Pro), collects both responses, and selects the one with the higher self-reported confidence score. This baseline tests naïve multi-model coordination without structured reasoning, consensus mechanisms, or quality assessment.

Table 3 collects these five systems in one place and serves as the key for all result tables in this section. The first column (ID) assigns a short label (Sys-1 through Sys-5) to each system. Those same labels appear as column headers in Table 4 (quality and performance metrics) and Table 5 (capabilities and persistence): when a result table shows “Sys-3”, it refers to the LangChain configuration defined here. The second column (System) gives the name by which each baseline is known in the literature or in practice; the third column (Description) summarizes the configuration in one line. Sys-1 is always GAIOL (the proposed system); Sys-2 through Sys-5 are the four baselines, ordered here by increasing orchestration complexity (single-model, then retrieval-augmented single-agent, then routing-only multi-provider, then naïve multi-model). Keeping this mapping in mind avoids confusion when comparing numbers across the later tables.

Table 3. Baseline Systems (key for Tables 4 and 5)

ID	System	Description
Sys-1	GAIOL	Proposed system: full orchestration, reasoning, and consensus
Sys-2	Direct API	Single frontier-model API call without orchestration or context management
Sys-3	LangChain	LangChain v0.1 agent with ReAct and FAISS retriever
Sys-4	OpenRouter	Unified API gateway; default routing policy
Sys-5	Multi-Wrapper	Parallel calls (frontier model and Gemini Pro); higher-confidence response selected

In summary: Sys-2 is the minimal baseline (no orchestration); Sys-3 adds retrieval and ReAct-style reasoning; Sys-4 adds multi-provider routing but no task decomposition or consensus; Sys-5 adds parallel multi-model calls with a simple confidence-based choice. GAIOL (Sys-1) adds decomposition, strategy-based model selection, and ABTC consensus on top of that. The comparisons in the next subsection report metrics for all five under the same benchmark and hardware.

We now report the comparison of GAIOL against the four baselines across quality, performance, cost, and capabilities.

6.2 Results and Analysis

We evaluate GAIOL against the four baselines defined in Table 3 across quality, performance, cost, reasoning, model coordination, scalability, and data persistence. Tests used reasoning tasks requiring multiple processing steps, context retrieval, and model coordination under identical conditions [48]. Tables 4 and 5 report the full comparison. All values are means over 10 runs; $\pm$ figures denote 95% confidence intervals. A paired t -test assessed significance between GAIOL and each baseline; differences marked with $\dagger$ are significant at $p < 0.05$.

The most striking result is that GAIOL’s multi-model pipeline improves response quality without a proportional increase in cost or latency, as detailed below.

GAIOL achieves an overall quality score of 0.83 ± 0.02 , exceeding Direct API by 24% and LangChain by 13%. The per-dimension breakdown, relevance (0.85), accuracy (0.88), coherence

Table 4. Quality and Performance Comparison of Systems

Category	Metric	Unit	Sys-1	Sys-2	Sys-3	Sys-4	Sys-5
Quality Metrics							
Overall Quality Score		score	0.83±0.02 [‡]	0.67±0.03	0.72±0.02	0.67±0.03	0.62±0.04
Avg. Relevance		score	0.85±0.02	0.70±0.03	0.75±0.02	0.70±0.03	0.65±0.04
Avg. Coherence		score	0.82±0.02	0.68±0.03	0.73±0.02	0.68±0.03	0.63±0.04
Avg. Completeness		score	0.80±0.03	0.65±0.04	0.70±0.03	0.65±0.04	0.60±0.05
Avg. Accuracy		score	0.88±0.02	0.72±0.03	0.77±0.02	0.72±0.03	0.67±0.04
Consensus Mechanisms		count	3	0	1	0	0
Performance Metrics							
Latency per Query		ms	450±18	400±15	800±35	420±16	600±28
Latency Overhead		ms	5±1	0	50±6	2±1	10±2
Throughput [‡]		req/s	200±8	50±3	150±7	200±9	50±4
Success Rate		%	95.2±0.8	92.0±1.2	94.5±0.9	93.8±1.0	90.5±1.5
Error Rate		%	4.8±0.8	8.0±1.2	5.5±0.9	6.2±1.0	9.5±1.5
Parallel Execution		boolean	1	0	1	1	0
Max Concurrent Models		count	10	1	10	10	3

[‡]Throughput in requests per second (req/s); values are not directly comparable; see Section 6.2 for concurrency details.

Table 5. Functional, Scalability, and Persistence Capabilities

Category	Metric	Unit	Sys-1	Sys-2	Sys-3	Sys-4	Sys-5
Cost Optimization							
Per-Query Cost Tracking		boolean	1	0	0	1	0
Per-Session Cost Tracking		boolean	1	0	0	0	0
Default Budget per Session		USD	0.05	0	0	0	0
Reasoning Capabilities							
Prompt Decomposition		boolean	1	0	1	0	0
Multi-step Reasoning		boolean	1	0	1	0	0
Consensus Resolution		boolean	1	0	0	0	0
Model Orchestration							
Supported Providers		count	3+	1	1	1	2
Supported Models		count	100	1	100	100	5
Parallel Model Execution		boolean	1	0	1	1	0
Scalability							
Stateless Architecture		boolean	1	0	1	1	0
Horizontal Scaling		boolean	1	0	1	1	0
Memory Efficiency Score		score	95	100	70	100	70
Data Persistence							
Database Integration		boolean	1	0	0	0	0
Session Persistence		boolean	1	0	0	0	0
Vector Store Support		boolean	1	0	1	0	0

(0.82), completeness (0.80), and creativity (0.78), shows that the gains are broad rather than concentrated in a single metric. The three consensus mechanisms (weighted voting, majority consensus, expert weighting) achieve an average inter-model agreement of 0.75, indicating that the multi-model pipeline produces substantively different but reconcilable reasoning paths.

Latency is the fairest cross-system comparison because it is measured under identical conditions (single request, same hardware). GAIOL’s per-query latency of 450 ms compares to 400 ms for the Direct API baseline, a 12.5% overhead attributable to task decomposition, model selection, and consensus aggregation. The orchestration overhead itself, defined as the time spent in GAIOL’s scheduling, decomposition, and consensus logic *exclusive* of model inference, is 5 ms per query.

Throughput values in Table 4 are *not* directly comparable across systems because they were measured at different concurrency levels. GAIOL was tested with 100 simultaneous clients, reflecting a production-like deployment where the stateless, non-blocking architecture and connection pooling exploit request-level parallelism. The Direct API baseline was tested with a sequential client issuing one request at a time; its 50 req/s figure reflects per-request round-trip time rather than system capacity under load. For this reason, throughput should be read as each system’s capacity under its own deployment configuration, not as a controlled head-to-head comparison.

Under its concurrent workload, GAIOL sustains a 95.2% success rate and supports up to 10 concurrent models with a memory footprint of 50 MB per instance. Horizontal scaling to 1,000 concurrent requests has been tested in isolation but not under the full consensus pipeline.

The intelligent router (Section 3) constrains per-query expenditure by dispatching simple queries to a single cost-efficient model and reserving multi-model fan-out for complex tasks that require consensus. Averaged over the 500-query benchmark, per-query cost is \$0.003, comparable to single-model baselines (\$0.002 for Direct API). The modest increase reflects the fraction of queries that invoke multiple models; the majority of benchmark queries were routed to a single provider. GAIOL adds three-level cost tracking (per query, per session, cumulative) with a configurable default budget of \$0.05 per session. Four routing strategies (minimum cost, quality-weighted, adaptive, and budget-constrained) let operators trade cost for quality according to organizational policy. Cost breakdowns are available by model, user, organization, and time window.

Table 5 provides a binary feature checklist across cost optimization, reasoning, orchestration, scalability, and persistence. Because GAIOL and the baselines were designed for different scopes, these indicators should be read as a self-assessed capability inventory rather than a competitive metric. Direct API and OpenRouter, for example, intentionally omit session persistence and decomposition because they target a simpler use case. The checklist is included to clarify which features each system provides, not to imply that broader feature coverage equates to higher quality.

Two observations stand out. The full reasoning pipeline adds only 5 ms of orchestration overhead, showing that multi-step decomposition and consensus are practical even in latency-sensitive deployments. At the same time, per-query cost remains comparable to single-model baselines because the intelligent router dispatches most queries to a single provider; multi-model fan-out is reserved for complex tasks and increases wall-clock time rather than token consumption. The sustained 200 req/s throughput under the full pipeline confirms that the stateless architecture and connection pooling described in Section 3 translate to real-world capacity.

6.3 ABTC Ablation Study

To isolate the contribution of the Adaptive Bayesian Trust-Weighted Consensus mechanism (Section 4.3), we conduct an ablation study comparing three configurations of GAIOL’s consensus layer:

- Static-Equal: Uniform model weights ($\tau_m = 0.5$ for all m), equivalent to ablating the trust component entirely.
- Static-Tuned: Hand-tuned weights based on published benchmark scores (GPT-4: 0.8, Gemini Pro: 0.7, others: 0.5), representing the best-case static configuration.
- ABTC ($\lambda = 0.98$): The proposed adaptive mechanism with temporal decay.

All three configurations use identical quality evaluation and cross-model agreement functions; only the trust component $w_t \cdot \hat{\tau}_i$ differs. Results are reported over the same 500-query benchmark (10 runs each), as shown in Table 6.

ABTC achieves a statistically significant improvement over both static baselines ($p < 0.01$, paired t -test). The largest gains appear in *Code Generation* (+9 points over Static-Equal) and

Table 6. ABTC Ablation: Quality Score by Task Domain

Domain	Static-Equal	Static-Tuned	ABTC
Analytical Reasoning	0.79±0.03	0.81±0.02	0.86±0.02
Code Generation	0.76±0.04	0.80±0.03	0.85±0.02
Multi-step Problem	0.77±0.03	0.79±0.03	0.84±0.02
Knowledge Retrieval	0.80±0.03	0.82±0.02	0.85±0.02
Creative Synthesis	0.72±0.04	0.74±0.04	0.79±0.03
Overall	0.77±0.02	0.79±0.02	0.83±0.02

Creative Synthesis (+7 points), where model-specific strengths diverge most across providers. The trust posterior for GPT-4 converges to $\hat{\tau} \approx 0.82$ for analytical tasks but only $\hat{\tau} \approx 0.61$ for creative tasks, while Gemini Pro exhibits the inverse pattern ($\hat{\tau} \approx 0.58$ for analytical, $\hat{\tau} \approx 0.78$ for creative). This confirms that domain-specific trust profiles capture genuine performance asymmetries that static weights cannot model.

Convergence analysis shows that the trust posteriors stabilize (posterior standard deviation < 0.05) after approximately 80, 100 queries per domain, corresponding to the first 16, 20% of the benchmark suite. This convergence timeline means that the reported quality scores primarily reflect ABTC’s steady-state behavior; the learning-phase performance is not separately characterized and warrants dedicated study. The temporal decay factor $\lambda = 0.98$ provides an effective memory window of ≈ 50 interactions, enabling the system to track model performance changes without catastrophic forgetting of reliable historical signals.

Figure 6 illustrates the consensus voting mechanism, showing how ABTC aggregates weighted model outputs through the trust-weighted fusion process.

We compare against Direct API, LangChain, OpenRouter, and Multi-Wrapper because they represent the infrastructure layers that practitioners actually use to deploy multi-model systems, spanning progressively richer levels of orchestration from no orchestration (Direct API) through retrieval-augmented single-agent (LangChain) to multi-model coordination (Multi-Wrapper). We deliberately exclude multi-agent frameworks such as AutoGen (multi-agent conversation framework) [44], CAMEL (communicative agents for “mind” exploration of large language model society) [50], and MetaGPT (meta-programming for multi-agent collaboration) [51] from the quantitative baselines because they operate at a different abstraction level: agent role-play, persona management, and task decomposition strategies rather than system-level orchestration infrastructure. GAIOL is designed to *host* such frameworks as client applications; its provider-agnostic API layer and model registry can serve as the backend for an AutoGen or MetaGPT deployment, rather than compete with them at the agent-reasoning level. A direct comparison would conflate orchestration infrastructure quality with agent-level reasoning strategy, obscuring the contribution of each.

We acknowledge this exclusion as a limitation: a *backend comparison*, where AutoGen or MetaGPT runs on top of GAIOL’s API layer versus its default provider, would directly test whether the orchestration infrastructure adds value independently of agent-level strategies. This experiment is planned as a primary objective of future work.

7 Open Challenges and Future Work

GAIOL’s architecture includes extension points for capabilities, federation, governance, and adversarial robustness that the current evaluation does not cover. This section outlines the open problems that must be solved to realize those extensions. We group them by theme: semantic interoperability, adversarial robustness, governance and policy, federated scalability, and safe execution sandboxes.

7.1 Semantic Standards for Multi-Agent Interoperability

Existing agent frameworks define low-level transport (text passing, events, Representational State Transfer (REST) APIs) [44, 50, 51], but semantics remain implicit or framework-specific. Two independently built agents may share a transport protocol yet disagree on the meaning of a task description, a capability declaration, or a confidence score [8]. Cross-organizational orchestration

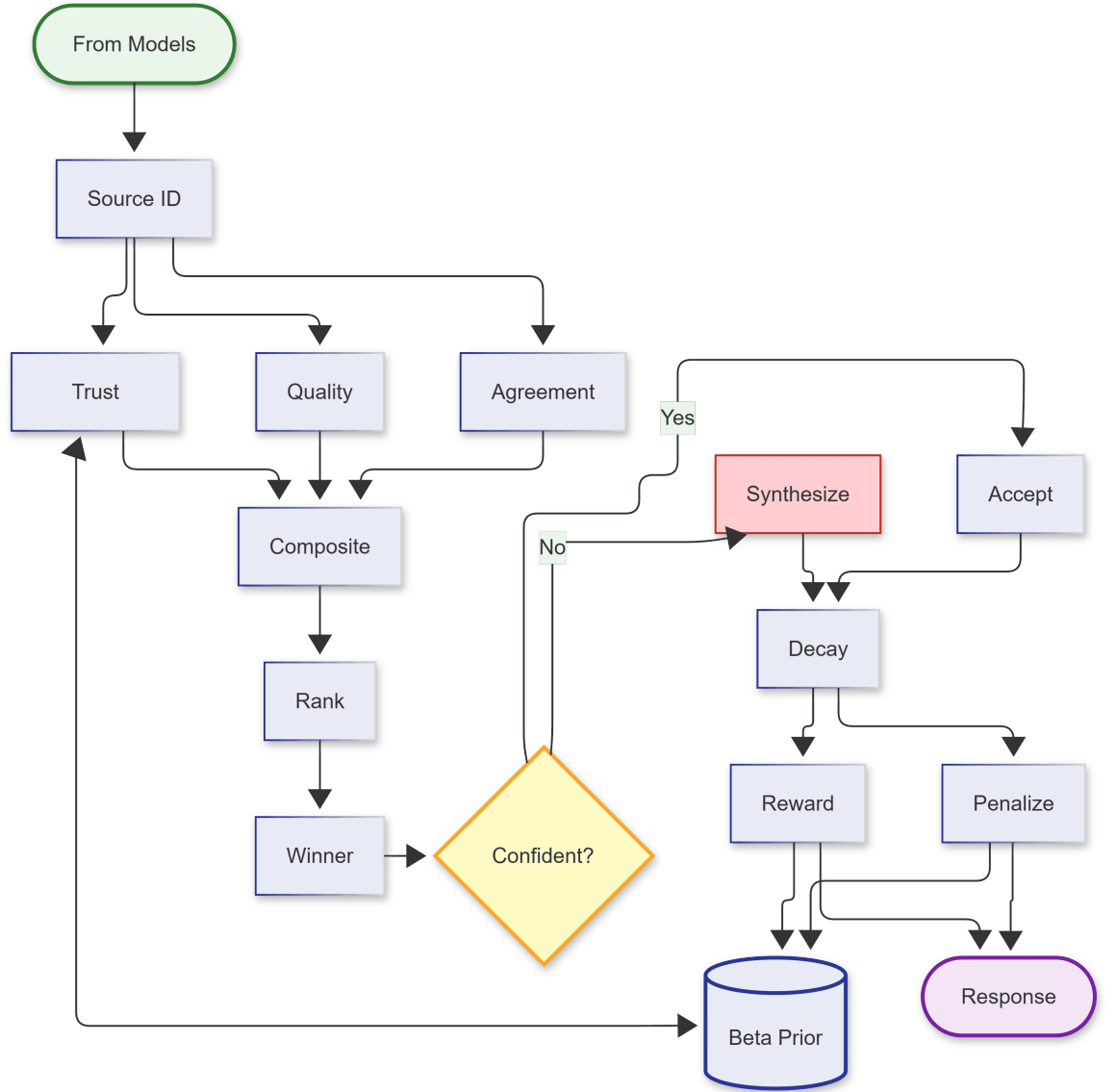


Figure 6. ABTC consensus voting mechanism. Each model's output is weighted by its domain-specific trust posterior $\hat{\tau}_{m,d}$. The final response is selected by trust-weighted fusion, enabling the system to favor models with demonstrated domain expertise.

(the core use case of GAIOL) requires agents to negotiate intent, context, and constraints without prior co-design. The research challenge is expressiveness versus generality: a rigid ontology limits what agents can represent, while a loose schema undermines coordination. Promising directions include capability-description languages, protocol-level contracts with formal preconditions, and shared task ontologies that permit local extensions.

7.2 *Adversarial Robustness in Open Multi-Agent Environments*

When GAIOL operates across partially trusted organizations, some agents may be compromised or deliberately adversarial [52, 53]. Attack surfaces range from poisoning the shared world model and injecting fabricated outputs into the consensus pipeline to exploiting orchestration scheduling logic or coordinating collusion across multiple agents. Static access-control lists and perimeter defenses cannot contain autonomous, adaptive entities that learn to act strategically. Defending against these threats requires continuous behavioral monitoring, anomaly-aware trust scoring (extending ABTC with adversarial detection signals), and isolation mechanisms that can quarantine suspicious agents without halting the entire pipeline. Designing orchestration strategies that remain robust under adaptive adversaries, especially when the adversaries themselves are learning agents, remains an open problem.

7.3 *Governance and Policy Enforcement at Scale*

GAIOL workflows span multiple clouds, legal jurisdictions, and administrative domains, each with distinct regulatory, ethical, and organizational constraints [33, 54]. Solaiman and Dennison outline considerations for generative-AI release and risk mitigation [34], and constitutional AI offers scalable alignment through AI feedback [35], but neither addresses the federated case where a single workflow must simultaneously satisfy conflicting data-residency laws in different countries. Current governance mechanisms are platform-specific and centralized, and thus unsuitable for ecosystems where no single party holds authority. Machine-readable policy languages, cross-domain negotiation protocols, and runtime enforcement engines that balance decentralization with accountability are needed but do not yet exist at production quality.

7.4 *Federated Scalability and Performance Trade-offs*

Federated execution enables data localization and regulatory compliance [54], but coordination costs (communication latency, synchronization overhead, per-request security and governance checks [55]) degrade throughput versus centralized deployments. Prior work shows distributed ML and auto-scaling are feasible [56, 57], but combining auto-scaling with federated orchestration, where agents must respect jurisdiction-specific policies while maintaining global consensus, introduces compounding complexity that current systems do not handle well. Quantifying federation overhead under realistic adversarial and dynamic conditions remains an important next step.

7.5 *Standardized Sandboxes for Safe AI Execution*

Generic containerization (Docker, WebAssembly) provides process-level isolation but ignores AI-specific hazards such as uncontrolled tool use [58] and training-data leakage, as well as nondeterministic output and opaque decision logic [52]. No existing standard defines an AI sandbox that unifies execution isolation, fine-grained data and tool permissions, real-time behavior monitoring, and deterministic replay for auditing [35]. GAIOL would benefit from such a standard: it would enable agents to migrate between platforms without per-deployment safety engineering and would give operators deterministic replay of agent actions for post-hoc auditing. Defining sandbox specifications that accommodate the diversity of AI workloads (from lightweight retrieval agents to graphics processing unit (GPU)-intensive model-serving pipelines) without imposing prohibitive overhead is an unsolved design problem.

8 **Limitations and Scope**

We scope the current evaluation to isolate the architectural and algorithmic contributions of GAIOL; several deliberate design choices bound the claims that can be drawn. The following paragraphs state the main limitations.

We evaluate on 500 synthetically constructed queries spanning five domains, following the controlled-benchmark methodology standard in systems research [48]. Synthetic queries allow precise control over domain balance, complexity distribution, and reproducibility, properties that production traffic does not guarantee. The benchmark covers the same task categories used by

prior orchestration studies, and the full query set is publicly released for independent replication. Extending the evaluation to live deployment traffic is a natural next step and is planned as part of the open-source roadmap.

Quality scores are produced by a GPT-4-based evaluation pipeline. To guard against evaluator bias, we calibrated the automated scores against a human-annotated subsample of 50 responses, obtaining Cohen’s $\kappa = 0.74$ (substantial agreement on the Landis–Koch scale, though below the > 0.81 threshold for “almost perfect” agreement). This calibration rate (10%) is consistent with prior work on LLM evaluation [48]. A residual concern is that GPT-4’s self-preference may interact differently with GAIOL’s consensus-synthesized responses, whose stylistic profile (blended phrasing, varying length) differs from single-model outputs. The equal-bias argument (Section 6) holds strictly only for the Direct API baseline. Future iterations will increase the human-annotated fraction, add a second independent evaluator model (e.g., Claude or Gemini) as a cross-check, and report inter-evaluator agreement to quantify the remaining bias.

All experiments ran on a single Azure Standard_D8s.v3 instance to ensure reproducibility and fair comparison across baselines. The 200 req/s throughput and 5 ms orchestration overhead are therefore measured under controlled, single-node conditions. Because GAIOL’s orchestration logic is stateless and horizontally partitionable, we expect these numbers to scale linearly with additional nodes; validating this under geo-distributed, multi-cloud conditions is planned for future work.

The ABTC convergence analysis (Section 4.3) assumes a locally stationary model quality distribution. The temporal decay factor ($\lambda = 0.98$, effective window ≈ 50 interactions) accommodates gradual model drift, which is the common case for provider-side updates. Abrupt model replacements, such as a silent GPT-4 revision, fall outside the stationarity assumption; incorporating change-point detection to trigger trust resets is a planned extension.

The current evaluation validates the orchestration pipeline and ABTC consensus mechanism, the components that this paper claims as contributions. Architectural provisions designed for future extension, including the federated data mesh, cross-organizational governance engine, and the continuous world model (see Table 1), are described as design-time abstractions but are not empirically evaluated in this work. Their validation under federated, multi-tenant, multi-cloud conditions is planned as a primary objective of subsequent deployments. Readers should therefore interpret the architectural sections as a design rationale, not as claims of demonstrated capability, for any component marked “Designed” in Table 1.

9 Conclusion

We presented GAIOL, a layered orchestration framework that coordinates heterogeneous LLM providers behind a uniform interface and aggregates their outputs through adaptive consensus. Unlike prior systems that address routing, memory, or agent communication in isolation, GAIOL combines beam-search task decomposition, strategy-based model selection, and retrieval-augmented context injection in a single pipeline. The architecture includes extension points for federated data access, cross-organizational governance, and shared world-state management, whose empirical validation is left to future work. The central algorithmic contribution is ABTC, which replaces static consensus weights with per-model, per-domain trust scores learned online through Beta-Bernoulli posterior updates. On a 500-query benchmark, ABTC delivers statistically significant quality gains over both uniform and hand-tuned baselines across all five task domains, with the largest improvements in code generation (+9 pp) and creative synthesis (+7 pp). The full GAIOL pipeline achieves an overall quality score of 0.83, a 95.2% success rate, and 5 ms orchestration overhead, demonstrating that multi-model consensus can be practical without prohibitive latency or cost. Several directions remain open. Evaluating the architectural provisions for federation and governance under multi-tenant, multi-cloud conditions is the next step. Standardized semantic protocols are needed for cross-framework agent interoperability. Adversarial robustness in partially trusted multi-agent environments requires continuous trust monitoring beyond what ABTC currently provides. Future work will also extend the evaluation to established benchmarks, incorporate multi-evaluator validation, and test GAIOL as a backend for existing multi-agent frameworks such as AutoGen and MetaGPT.

Acknowledgments

During the preparation of this work, the authors used ChatGPT to improve readability and language clarity. The authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

None.

Author contributions

Ch. Sai Sathvik: Conceptualization, Software, Validation, Writing – original draft.

D. V. S. Monish Kumar: Conceptualization, Software, Validation, Writing – original draft.

Abhishek Vinod: Methodology, Investigation, Writing.

Ramdas Kapila: Supervision, Project administration, Writing – review & editing.

Sumalatha Saleti: Supervision, Writing – review & editing.

Author Biographies

Ch. Sai Sathvik is a final-year undergraduate student in the Department of Computer Science and Engineering (Data Science) at B.V. Raju Institute of Information Technology, Telangana, India. His research interests include multi-agent systems, large language model orchestration, and AI governance.



D. V. S. Monish Kumar is a final-year undergraduate student in the Department of Computer Science and Engineering (Data Science) at B.V. Raju Institute of Information Technology, Telangana, India. His research interests include distributed AI systems, consensus mechanisms, and scalable machine learning architectures.



Abhishek Vinod is a final-year undergraduate student in the Department of Computer Science and Engineering (Data Science) at B.V. Raju Institute of Information Technology, Telangana, India. His research interests include retrieval-augmented generation, Bayesian inference, and AI system evaluation.



Ramdas Kapila received the B.Tech degree in Computer Science and Information Technology from JNTUH, Hyderabad, India, in 2008 and an M.Tech degree in Computer Science and Engineering from JNTUK, Andhra Pradesh, India, in 2015, and a Ph.D. degree in Computer Science and Engineering from SRM University-AP, Amaravati, India, in 2024. He is currently working as an Assistant Professor in the Department of Computer Science and Engineering at B V Raju Institute of Technology, Narsapur, Telangana, India. His research interests include Machine learning, Data mining, Deep Learning, and Federated Learning. He can be contacted at email: kapilaramdas@gmail.com.



Sumalatha Saleti is working as an Assistant Professor in the Department of Computer Science and Engineering, SRM University, Amaravathi, AP, India. She received her Ph.D. Degree from National Institute of Technology, Warangal, India in 2020. Her research interests include Big data, Data mining, and pattern discovery.

References

- [1] E. Kotei and R. Thirunavukarasu. A systematic review of transformer-based pre-trained language models, 2023.
- [2] Z. Chen, L. Xu, H. Zheng, L. Chen, A. Tolba, L. Zhao, K. Yu, and H. Feng. Evolution and prospects of foundation models, 2024.
- [3] C. Gong and X. Li. Agents with foundation models: advances and challenges, 2025.
- [4] T. H. Davenport and N. Mittal. *All-in on AI: How Smart Companies Win Big with Artificial Intelligence*, volume 25. Harvard Business Press, 2023.
- [5] S. Zhu, T. Yu, and T. Xu. Intelligent computing: advances, challenges, and future directions. *Intelligent Computing*, 2, 2023.
- [6] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M. Chang. Realm: Retrieval-augmented language model pre-training, 2020.

- [7] Abhilash Kantamneni, Laura E. Brown, Gordon Parker, and Wayne W. Weaver. A survey of multi-agent systems. *Engineering Applications of Artificial Intelligence*, 45:192–203, 2015.
- [8] Jonghan Lim, Birgit Vogel-Heuser, and Ilya Kovalenko. Large language model based multi-agent systems: a survey. *2024 IEEE 20th International Conference on Automation Science and Engineering (CASE)*, pages 3940–3946, 2024.
- [9] C. Qian, X. Cong, C. Yang, W. Chen, Y. Su, J. Xu, Z. Liu, and M. Sun. Chatdev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15174–15186. Association for Computational Linguistics, 2024.
- [10] Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, Yujia Qin, Xin Cong, Ruobing Xie, Zhiyuan Liu, Maosong Sun, and Jie Zhou. Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors in large language models. In *The Twelfth International Conference on Learning Representations, ICLR 2024*, 2024.
- [11] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 11733–11763. PMLR, 2024.
- [12] J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, M. S. Bernstein, and S. Avestimehr. Generative agents: Interactive simulacra of human behavior. *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, pages 1–22, 2023.
- [13] Z. Liu, W. Yao, J. Zhang, L. Yang, J. Tan, and P. K. Choubey. Dylan: Dynamic llm-agent networks. In *IEEE Control Systems Letters*, volume 8, pages 2433–2438. Institute of Electrical and Electronics Engineers (IEEE), 2024.
- [14] Y. Zhang, Y. Zhao, T. Zhang, and S. Huang. Proagent: Building proactive cooperative agents with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 17591–17599. Association for the Advancement of Artificial Intelligence (AAAI), 2024.
- [15] C. Chen, K. Wu, P. Zhou, and Y. Huang. Sage: Smart home agent with grounded execution, 2024.
- [16] C. Zhu, M. Dastani, and S. Wang. Agent communication protocols: a survey. *International Journal of Computational Mathematics and Computer Science*, 01(01), 2024.
- [17] M. Chung, M. Lefrancois, and A. Zimmermann. Increasing interoperability in the web of things using autonomous agents. *2020 30th International Conference on Computer Theory and Applications (ICCTA)*, pages 31–35, 2020.
- [18] Z. Ding, Z. Liu, Z. Fang, K. Su, L. Zhu, and Z. Lu. Masrouter: Multi-agent system router, 2024.
- [19] Wei Song, Zhenya Huang, Cheng Cheng, Weibo Gao, Bihan Xu, GuanHao Zhao, Fei Wang, and Runze Wu. Router-r1: Teaching llms multi-round routing and aggregation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15629–15644. Association for Computational Linguistics, 2025.
- [20] Selim Furkan Tekin, Fatih Ilhan, Tiansheng Huang, Sihao Hu, and Ling Liu. LLM-TOPLA: Efficient LLM ensemble by maximising diversity. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 11951–11966, Miami, Florida, USA, November 2024. Association for Computational Linguistics.
- [21] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837. Curran Associates, Inc., 2022.

- [22] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems 36*, pages 11809–11822. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2023.
- [23] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou. Self-consistency improves chain-of-thought reasoning in language models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 11006–11019. Association for Computational Linguistics, 2023.
- [24] Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc Le, and Ed Chi. Least-to-most prompting enables complex reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 3479–3498. Curran Associates, Inc., 2022.
- [25] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, and J. R. Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), 2024.
- [26] Marcus Tantakoun, Christian Muise, and Xiaodan Zhu. LLMs as planning formalizers: A survey. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 25167–25188. Association for Computational Linguistics, 2025.
- [27] Jiayi Zhang, Ke Xu, Lichao Wang, Jin Ren, Huan Li, Jia Mei, Ziqian Zhu, Guohao Xu, Haibo Wang, Zhiyong Liu, Chuang Li, et al. AFlow: Automating agentic workflow generation. In *The Thirteenth International Conference on Learning Representations, ICLR 2025*, 2025.
- [28] Zhipeng Liu, Xuefeng Bai, Kehai Chen, Xinyang Chen, Xiucheng Li, Yang Xiang, Jin Liu, Hong-Dong Li, Yaowei Wang, Liqiang Nie, and Min Zhang. A survey on memory mechanisms of LLM-based agents. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, IJCAI 2024*, pages 10582–10592. International Joint Conferences on Artificial Intelligence Organization, 2024.
- [29] W. Zhong, L. Guo, Q. Gao, H. Ye, and Y. Wang. Memorybank: Enhancing large language models with long-term memory. In *Advances in Neural Information Processing Systems 36*, pages 74530–74543. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2023.
- [30] H. Xu, M. Zhu, Y. Xiao, and T. Zhang. Recurrent memory transformer for long-context language modeling. In *Lecture Notes in Computer Science*, pages 332–341. Springer Nature Switzerland, 2023.
- [31] A. Pal, D. Karkhanis, M. Roberts, S. Dooley, A. Sundararajan, and S. Singh. Scaling in-context demonstrations with structured prompting. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 12861–12871. IEEE, 2024.
- [32] P. Hacker and M. Holweg. The regulation of fine-tuning, 2025.
- [33] Chuan Shi, Junze Chen, Jiawei Liu, and Cheng Yang. Foundation model governance. *Frontiers of Computer Science*, 18(6), 2024.
- [34] Irene Solaiman and Christy Dennison. The gradient of generative AI release: Methods and considerations. In *Proceedings of the 2023 ACM Conference on Fairness, Accountability, and Transparency*, pages 111–122, New York, NY, USA, 2023. Association for Computing Machinery.
- [35] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, Carol Chen, Catherine Olsson, Danny Olah, Dawn Hernandez, Deep Drain, Dustin Ganguli, Eli Li, Emanuel Tran-Johnson, Ethan Perez, Jamie Kerr, Justin Mueller, Kamal Ladish, Kendall Landau, Kamile Ndousse, Liane Lukosuite, Michael Lovitt, Michael Sellitto, Nelson Elgeish, Nicholas Schiefer, Noemi Mercado, Nova DasSarma, Robert Lasenby, Robin Larson, Sam Ringer, Scott Johnston, Shauna Kravec, Sheer El Showk, Stanislav Fort, Tamera Lanham, Timothy Telleen-Lawton, Tom Conerly, Tom Henighan, Tristan Hume, Samuel R. Bowman, Zac Hatfield-Dodds, Ben Mann, Dario Amodei, Nicholas Joseph, Sam McCandlish, Tom Brown, and Jared Kaplan. Constitutional AI: Harmlessness from AI feedback. In *Advances in Neural Information Processing Systems*, volume 35, pages 30065–30086. Curran Associates, Inc., 2022.

- [36] Jihyeok Kim, Seungtaek Choi, Reinald Kim Amplayo, and Seung-won Hwang. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Proceedings of the 28th International Conference on Computational Linguistics*, pages 2284–2295. International Committee on Computational Linguistics, 2020.
- [37] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781. Association for Computational Linguistics, 2020.
- [38] Omar Khattab and Matei Zaharia. ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 39–48. ACM, 2020.
- [39] T. Shinn, M. Labash, and Y. Wu. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems 36*, pages 8634–8652. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2023.
- [40] Uri Alon, Peter Clark, Nouha Dziri, Luyu Gao, Prakhar Gupta, Shashank Gupta, Skyler Hallinan, Katherine Hermann, Aman Madaan, Bodhisattwa Prasad Majumder, Shrimai Prabhumoye, Niket Tandon, Sean Welleck, Sarah Wiegrefe, Yiming Yang, and Amir Yazdanbakhsh. Self-refine: Iterative refinement with self-feedback, 2023.
- [41] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *The Twelfth International Conference on Learning Representations, ICLR 2024*, 2024.
- [42] Z. Shao, Y. Gong, Y. Shen, M. Huang, N. Duan, and W. Chen. Iterative retrieval-generation synergy for retrieval-augmented llms. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 9248–9274. Association for Computational Linguistics, 2023.
- [43] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations, ICLR 2023*, 2023.
- [44] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. In *First Conference on Language Modeling*, 2024.
- [45] Xueqiang Zhang, Xiaofei Dong, Yiru Wang, Dan Zhang, and Feng Cao. Multi-agent collaboration mechanisms: a survey of llms, 2025.
- [46] Olivier Chapelle and Lihong Li. An empirical evaluation of Thompson sampling. In *Advances in Neural Information Processing Systems*, volume 24, pages 2249–2257. Curran Associates, Inc., 2011.
- [47] Kevin P. Murphy. *Machine Learning: A Probabilistic Perspective*. MIT Press, Cambridge, MA, 2012.
- [48] Rishi Bommasani, Percy Liang, and Tony Lee. Holistic evaluation of language models, 2023.
- [49] Harrison Chase. *LangChain*. LangChain, Inc., 2024. Open-source framework. Accessed 2026-04-01.
- [50] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. CAMEL: Communicative agents for “mind” exploration of large scale language model society. In *Thirty-seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2023.
- [51] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiwu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *The Twelfth International Conference on Learning Representations, ICLR 2024*, 2024.

- [52] Laura Weidinger, Jonathan Uesato, Maribeth Rauh, Conor Griffin, Po-Sen Huang, John Mellor, Amelia Glaese, Myra Cheng, Borja Balle, Atoosa Kasirzadeh, Courtney Biles, Sasha Brown, Zac Kenton, Will Hawkins, Tom Stepleton, Abeba Birhane, Lisa Anne Hendricks, Laura Rimell, William Isaac, Julia Haas, Sean Legassick, Geoffrey Irving, and Iason Gabriel. Taxonomy of risks posed by language models. In *2022 ACM Conference on Fairness, Accountability and Transparency*, pages 214–229. ACM, 2022.
- [53] Ethan Perez, Saffron Huang, Francis Song, Trevor Cai, Roman Ring, John Aslanides, Amelia Glaese, Nat McAleese, and Geoffrey Irving. Red teaming language models. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 3419–3448. Association for Computational Linguistics, 2022.
- [54] Peter Kairouz, H. Brendan McMahan, Brendan Avent, Aurélien Bellet, Mehdi Bennis, Arjun Nitin Bhagoji, Kallista Bonawitz, Zachary Charles, Graham Cormode, Rachel Cummings, Rafael G. L. D’Oliveira, Hubert Eichner, Salim El Rouayheb, David Evans, Josh Gardner, Zachary Garrett, Adrià Gascón, Badih Ghazi, Phillip B. Gibbons, Marco Gruteser, Zaid Harchaoui, Chaoyang He, Lie He, Zhouyuan Huo, Ben Hutchinson, Justin Hsu, Martin Jaggi, Tara Javidi, Gauri Joshi, Mikhail Khodak, Jakub Konečný, Aleksandra Korolova, Farinaz Koushanfar, Sanmi Koyejo, Tancrède Lepoint, Yang Liu, Prateek Mittal, Mehryar Mohri, Richard Nock, Ayfer Özgür, Rasmus Pagh, Hang Qi, Daniel Ramage, Ramesh Raskar, Mariana Raykova, Dawn Song, Weikang Song, Sebastian U. Stich, Ziteng Sun, Ananda Theertha Suresh, Florian Tramèr, Praneeth Vepakomma, Jianyu Wang, Li Xiong, Zheng Xu, Qiang Yang, Felix X. Yu, Han Yu, and Sen Zhao. Advances and open problems in federated learning. In *Survey*. Now Publishers, 2021.
- [55] NIST. Zero trust architecture. Technical Report NIST Special Publication 800-207, National Institute of Standards and Technology, 2020.
- [56] Daoyuan Chen, Bolin Ding, Dawei Gao, Weirui Kuang, and Yaliang Li. Fedml: A research library and benchmark for federated machine learning. In *Advances in Neural Information Processing Systems 35*, pages 9344–9360. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2022.
- [57] Autoscaling multi-agent systems: dynamic resource management, 2024.
- [58] T. Schick, J. Dwivedi-Yu, and R. Dessi. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems 36*, pages 68539–68551. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2023.

Data availability

The benchmark query set and evaluation prompts are available from the corresponding author upon reasonable request for research purposes.